Untitled-1.cpp

```cpp
#include <cuda_runtime.h>
// 1. Warp 级别规约 (32个线程内的极速求和)
__inline__ __device__ float warpReduceSum(float val) {
    // 每次跨度减半，利用 __shfl_down_sync 让线程读取其他线程的寄存器
    for (int offset = warpSize / 2; offset > 0; offset /= 2) {
        val += __shfl_down_sync(0xffffffff, val, offset);
    }
    return val;
}
// 2. Block 级别规约 (整合多个 Warp 的结果)
__inline__ __device__ float blockReduceSum(float val) {
    static __shared__ float shared[32]; // 最多支持 1024 线程/Block (32个Warp)
    int lane = threadIdx.x % warpSize;  // 当前线程在 Warp 中的索引
    int wid = threadIdx.x / warpSize;   // 当前线程所属的 Warp ID

    val = warpReduceSum(val);           // 第一步：每个 Warp 内部先规约
    if (lane == 0) shared[wid] = val;   // 第二步：每个 Warp 的总和写入共享内存
    __syncthreads();                    // 同步，确保所有 Warp 都写完了
    // 第三步：让第一个 Warp 去读取共享内存中的中间结果，再做一次 Warp 规约
    val = (threadIdx.x < blockDim.x / warpSize) ? shared[lane] : 0;
    if (wid == 0) val = warpReduceSum(val);
    return val;
}
// 3. 全局 Kernel (支持任意大小的 N)
__global__ void reduceSumModernKernel(float* d_in, float* d_out, int N) {
    float sum = 0;
    // 网格跨步循环 (Grid-stride loop)：每个线程可以处理多个数据，无惧数组超大
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < N; i += blockDim.x * gridDim.x) {
        sum += d_in[i];
    }
    sum = blockReduceSum(sum); // Block 内规约
    // 第四步：Block 的0号线程将结果累加到全局内存 (Grid 级别规约)
    if (threadIdx.x == 0) atomicAdd(d_out, sum);
}
```